

Let's fix each of these 3 specific issues step-by-step so your setup works reliably.

1. Pressure Changes & Height Calculation (Up vs Down)

At sea level, 1 Hectopascal (100 Pa) of **pressure drop corresponds to approximately 8.5 to 9 meters of elevation gain**.

Here is how much pressure change you should expect when moving the sensor vertically:

Physical Movement	Expected Pressure Change (Pa)	Expected Height (Z)
Lifting board up by 1 meter (desk to head height)	Decreases by $\approx$ 11 to 12 Pa	+1.00 m
Lifting board up by 3 meters (Climbing 1 floor)	Decreases by $\approx$ 35 to 40 Pa	+3.00 m (Floor 1)
Lowering board down by 1 meter	Increases by $\approx$ 11 to 12 Pa	-1.00 m

Why Pressure Might Be Unstable:

- **Air Currents & Drafts:** The ICP-10111 is sensitive enough to detect wind from ceiling fans or AC vents. Keep it away from direct breezes.
- **Light Sensitivity:** Strong, direct bright sunlight or direct flashlights hit the open lid hole of the MEMS element and can skew capacitive readings.

2. Why Direction (Compass) Was Inaccurate

The magnetometer output (M_x, M_y) directly depends on how the sensor is held.

- **Tilt Sensitivity:** If the board is held tilted instead of horizontal, gravity skews M_x and M_y , causing large direction errors. Keep the sensor **flat/horizontal** relative to the ground.
- **Indoor Hard-Iron Interference:** Laptops, metal desks, and USB power banks introduce magnetic fields. You must complete the **Figure-8 rotation in the air (away from metal objects)** during the 7-second calibration phase.

3. Why Steps Weren't Detecting

In the strict mode, the code requires **continuous swinging motion over a 15-sample buffer** (0.3 seconds) to trigger the REAL WALKING state. If you are testing by holding the board in your hand and taking isolated single steps or moving slowly, the rolling variance (σ) won't reach the required threshold.

Diagnostics & Fully Working Updated Code

Here is the updated code. It includes **diagnostic printouts** so you can see your live raw pressure (Pa), rolling variance, and heading, making it easy to observe what the sensor is reading in real time:

Python

```

# =====

# DIAGNOSTIC & TRACKING PROGRAM (HARD I2C)

# Fixes: Sensitivities adjusted, live raw values printed for troubleshooting

# =====

from machine import Pin, I2C

import utime, math

# --- HARD I2C CONFIGURATION ---

I2C_ID = 0

SDA_PIN = 0

SCL_PIN = 1

I2C_FREQ = 100000

ACC_ADDRESS = 0x19

MAG_ADDRESS = 0x1E

BARO_ADDRESS = 0x63

# Registers

CTRL_REG1_A = 0x20

CTRL_REG4_A = 0x23

OUT_X_L_A = 0x28

CRA_REG_M = 0x00

CRB_REG_M = 0x01

MR_REG_M = 0x02

OUT_X_H_M = 0x03

# Tuning Parameters

STRIDE_LENGTH_M = 0.50

STEP_COOLDOWN_MS = 350

ALPHA = 0.35

FLOOR_HEIGHT_M = 3.0

```

```
# Navigation State
```

```
x_bias, y_bias, z_bias = 0.0, 0.0, 0.0
```

```
mag_x_bias, mag_y_bias = 0.0, 0.0
```

```
baseline_pressure = 0.0
```

```
total_steps = 0
```

```
total_distance = 0.0
```

```
pos_x, pos_y, pos_z = 0.0, 0.0, 0.0
```

```
current_floor = 0
```

```
last_step_time = 0
```

```
last_heading = 0.0
```

```
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```
def write_reg(addr, reg, val):
```

```
    try: i2c.writeto_mem(addr, reg, bytes([val]))
```

```
    except: pass
```

```
def read_regs(addr, reg, length):
```

```
    try: return i2c.readfrom_mem(addr, reg, length)
```

```
    except: return None
```

```
def signed_12bit(val):
```

```
    val >>= 4
```

```
    if val >= 2048: val -= 4096
```

```
    return val
```

```
def signed_16bit(val):
```

```
    if val >= 32768: val -= 65536
```

```
    return val
```

```
# --- ICP-10111 DRIVER ---
```

```
class ICP10111:
```

```
    def __init__(self, i2c_bus):
```

```
        self.i2c = i2c_bus
```

```
        self.c1, self.c2, self.c3, self.c4 = 0, 0, 0, 0
```

```
        self.calibrated = False
```

```
        self.p_Pa_calib = [45000.0, 80000.0, 105000.0]
```

```
        self.LUT_lower = 3.5 * (1 << 20)
```

```
        self.LUT_upper = 11.5 * (1 << 20)
```

```
        self.quadr_factor = 1.0 / 16777216.0
```

```
        self.offst_factor = 2048.0
```

```
    def init_sensor(self):
```

```
        try:
```

```
            self.i2c.writeto(BARO_ADDRESS, bytes([0x80, 0x5D]))
```

```
            utime.sleep_ms(10)
```

```
            self.i2c.writeto(BARO_ADDRESS, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))
```

```
            utime.sleep_ms(5)
```

```
            otp = []
```

```
            for _ in range(4):
```

```
                self.i2c.writeto(BARO_ADDRESS, bytes([0xC7, 0xF7]))
```

```
                data = self.i2c.readfrom(BARO_ADDRESS, 3)
```

```
                if not data or len(data) < 3 or (data[0] == 0 and data[1] == 0):
```

```
                    return False
```

```
                raw_val = (data[0] << 8) | data[1]
```

```
                if raw_val >= 32768: raw_val -= 65536
```

```
                otp.append(raw_val)
```

```
            self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]
```

```

        self.calibrated = True

        print(f"[ICP-10111] Ready! OTP: C1={self.c1}, C2={self.c2}, C3={self.c3}, C4={self.c4}")

        return True

    except Exception as e:

        print("[!] Barometer Error:", e)

        return False

def _calc_conversion_constants(self, p_Pa, p_LUT):

    denom_c = (p_LUT[2] * (p_Pa[0] - p_Pa[1]) +
               p_LUT[0] * (p_Pa[1] - p_Pa[2]) +
               p_LUT[1] * (p_Pa[2] - p_Pa[0]))

    if abs(denom_c) < 1e-6: return None, None, None

    C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +
         p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) +
         p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / denom_c

    denom_a = (p_LUT[0] - p_LUT[1])

    if abs(denom_a) < 1e-6: return None, None, None

    A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / denom_a
    B = (p_Pa[0] - A) * (p_LUT[0] + C)

    return A, B, C

def get_pressure(self):

    if not self.calibrated:

        if not self.init_sensor(): return None

    try:

        self.i2c.writeto(BARO_ADDRESS, bytes([0x68, 0x25]))

        utime.sleep_ms(15)

```

```

res = self.i2c.readfrom(BARO_ADDRESS, 9)

if not res or len(res) < 9: return None

if (res[0] == 0 and res[1] == 0) or (res[0] == 0xFF and res[1] == 0xFF): return None

T_LSB = (res[0] << 8) | res[1]

p_LSB = (res[3] << 16) | (res[4] << 8) | res[6]

t = float(T_LSB - 32768)

s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor
s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor
s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor

A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])

if A is None or B is None or C is None: return None

denom_p = C + float(p_LSB)

if abs(denom_p) < 1e-6: return None

pressure_pa = A + (B / denom_p)

return pressure_pa

except:

    return None

icp_baro = ICP10111(i2c)

# --- LSM303 DRIVERS ---

def init_lsm303():

    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # Accel 100Hz ODR

    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High Res

    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14) # Mag 30Hz ODR

```

```
write_reg(MAG_ADDRESS, CRB_REG_M, 0x20) # Gain
write_reg(MAG_ADDRESS, MR_REG_M, 0x00) # Continuous
```

```
def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)
    if data is None: return None
    return signed_12bit((data[1]<<8)|data[0]), signed_12bit((data[3]<<8)|data[2]),
    signed_12bit((data[5]<<8)|data[4])
```

```
def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
    if data is None: return None
    return signed_16bit((data[0]<<8)|data[1]), signed_16bit((data[2]<<8)|data[3]),
    signed_16bit((data[4]<<8)|data[5])
```

```
def calibrate_system():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias, baseline_pressure

    print("\n=====")
    print("STEP 1/2: ACCEL & BARO CALIBRATION (Keep board FLAT & STILL)")
    print("=====")
    sx, sy, sz, p_sum, count = 0, 0, 0, 0, 0

    while count < 50:
        acc = read_accel()
        p = icp_baro.get_pressure()
        if acc and p:
            sx += acc[0]; sy += acc[1]; sz += acc[2]
            p_sum += p; count += 1
        utime.sleep_ms(30)

    x_bias = sx / count
```

```

y_bias = sy / count
z_bias = (sz / count) - 1024.0
baseline_pressure = p_sum / count

print(f"-> Locked Baseline Pressure: {baseline_pressure:.2f} Pa")

```

```

print("\n=====")
print("STEP 2/2: MAGNETOMETER CALIBRATION (Rotate in Figure-8)")
print("=====")
min_x, max_x, min_y, max_y = 32767, -32768, 32767, -32768

```

```

start_time = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start_time) < 7000:
    m = read_mag()
    if m:
        if m[0] < min_x: min_x = m[0]
        if m[0] > max_x: max_x = m[0]
        if m[1] < min_y: min_y = m[1]
        if m[1] > max_y: max_y = m[1]
    utime.sleep_ms(20)

```

```

mag_x_bias = (max_x + min_x) / 2.0
mag_y_bias = (max_y + min_y) / 2.0
print(f"-> Mag Biases: X={mag_x_bias:.1f}, Y={mag_y_bias:.1f}\n")

```

```

def get_heading():
    m = read_mag()
    if m is None: return 0.0
    mx_c = m[0] - mag_x_bias
    my_c = m[1] - mag_y_bias
    heading = math.degrees(math.atan2(my_c, mx_c))

```



```
return heading + 360.0 if heading < 0 else heading
```

```
def get_cardinal(h):
```

```
    if h >= 337.5 or h < 22.5: return "NORTH"
```

```
    elif 22.5 <= h < 67.5: return "NE"
```

```
    elif 67.5 <= h < 112.5: return "EAST"
```

```
    elif 112.5 <= h < 157.5: return "SE"
```

```
    elif 157.5 <= h < 202.5: return "SOUTH"
```

```
    elif 202.5 <= h < 247.5: return "SW"
```

```
    elif 247.5 <= h < 292.5: return "WEST"
```

```
    else: return "NW"
```

```
def main():
```

```
    global total_steps, total_distance, last_step_time, pos_x, pos_y, pos_z
```

```
    global current_floor, last_heading
```

```
    init_lsm303()
```

```
    icp_baro.init_sensor()
```

```
    calibrate_system()
```

```
    smooth_accel = 0.0
```

```
    accel_buffer = []
```

```
    print(">>> TRACKING ACTIVE: WALK WITH NATURAL ARM SWING <<<\n")
```

```
    diag_counter = 0
```

```
    while True:
```

```
        now = utime.ticks_ms()
```

```
        acc = read_accel()
```

```
        if acc:
```

```

ax = acc[0] - x_bias
ay = acc[1] - y_bias
az = acc[2] - z_bias

raw_mag = math.sqrt(ax**2 + ay**2 + az**2)
smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)

accel_buffer.append(smooth_accel)
if len(accel_buffer) > 15: accel_buffer.pop(0)

if len(accel_buffer) == 15:
    avg_val = sum(accel_buffer) / 15.0
    variance = sum((v - avg_val)**2 for v in accel_buffer) / 15.0
    std_dev = math.sqrt(variance)
    p2p_delta = max(accel_buffer) - min(accel_buffer)

# 1. Read Pressure & Calculate Z Height
current_p = icp_baro.get_pressure()
if current_p is not None:
    pos_z = 44330.0 * (1.0 - math.pow((current_p / baseline_pressure), 0.1903))
    current_floor = int(round(pos_z / FLOOR_HEIGHT_M))

heading = get_heading()
cardinal = get_cardinal(heading)

# Diagnostic Output (Prints every 1 second while stationary)
diag_counter += 1
if diag_counter % 25 == 0:
    p_str = f"{current_p:.1f} Pa" if current_p else "N/A"
    print(f"[DIAGNOSTIC] Press: {p_str} | Z: {pos_z:+.2f}m | Heading: {cardinal}
({heading:.0f}°) | StdDev: {std_dev:.1f}")

```

```

# 2. Step Detection Condition

# Condition: Dynamic swing pulse + moderate variation
if (smooth_accel > (avg_val * 1.12)) and (std_dev > 1.2) and (p2p_delta < 120.0):
    time_diff = utime.ticks_diff(now, last_step_time)

    if time_diff > STEP_COOLDOWN_MS:
        total_steps += 1
        total_distance += STRIDE_LENGTH_M
        last_step_time = now

# 2D Position Trigonometry
rad = math.radians(heading)
pos_x += STRIDE_LENGTH_M * math.sin(rad)
pos_y += STRIDE_LENGTH_M * math.cos(rad)

print(f"*** STEP #{total_steps:03d} *** | "
      f"Heading: {cardinal} ({heading:.0f}°) | "
      f"Pos(X,Y): ({pos_x:.2f}m, {pos_y:.2f}m) | "
      f"Height(Z): {pos_z:+.2f}m | Floor: {current_floor}")

utime.sleep_ms(20)

if __name__ == "__main__":
    main()

```

////////////////////////////////

Here is your complete, end-to-end framework for **real-time AI/ML 3D Indoor Pedestrian Dead Reckoning (3D-PDR)** using Hard I^2C on the RT1064 EVK, LSM303DLHC, and the ICP-10111 Barometer.

Since you are testing in a real environment today, we will execute a **3-Step Process** to train the Decision Tree classifier with high precision:

STEP 1: Record Clean Training Data (MicroPython)

Flash this data logger script onto your RT1064 board. Run it and collect **3 separate datasets** from the REPL console by copying the printed lines into text files on your PC:

1. idle.csv: Hold board naturally while sitting/standing still (15 seconds).
2. walking.csv: Walk naturally with a rhythmic arm swing (30 seconds).
3. shaking.csv: Tap the desk, shake the USB cable, or move your hands wildly (15 seconds).

Python

```
# =====  
  
# STEP 1: AI DATA LOGGER (Run on RT1064 to generate CSVs)  
  
# =====  
  
from machine import Pin, I2C  
import utime, math  
  
i2c = I2C(0, freqs=100000, sda=Pin(0), scl=Pin(1))  
  
def write_reg(addr, reg, val):  
    try: i2c.writeto_mem(addr, reg, bytes([val]))  
    except: pass  
  
def read_regs(addr, reg, length):  
    try: return i2c.readfrom_mem(addr, reg, length)  
    except: return None  
  
def signed_12bit(val):  
    val >= 4  
    if val >= 2048: val -= 4096  
    return val  
  
write_reg(0x19, 0x20, 0x57) # Accel 100Hz  
write_reg(0x19, 0x23, 0x08) # High Res
```

```

print("--- DATA LOGGING ACTIVE: Copy REPL lines into CSV ---")

buffer = []

ALPHA = 0.30

smooth_accel = 0.0

while True:

    data = read_regs(0x19, 0x28 | 0x80, 6)

    if data:

        x = signed_12bit((data[1] << 8) | data[0])

        y = signed_12bit((data[3] << 8) | data[2])

        z = signed_12bit((data[5] << 8) | data[4])


        raw_mag = math.sqrt(x**2 + y**2 + z**2)

        smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)


        buffer.append(smooth_accel)

        if len(buffer) > 15: buffer.pop(0)


    if len(buffer) == 15:

        avg_val = sum(buffer) / 15.0

        variance = sum((v - avg_val)**2 for v in buffer) / 15.0

        std_dev = math.sqrt(variance)

        p2p_delta = max(buffer) - min(buffer)


        # Print features for CSV collection

        print(f'{smooth_accel:.2f},{avg_val:.2f},{std_dev:.2f},{p2p_delta:.2f}')


    utime.sleep_ms(20)

```

STEP 2: Train Model & Auto-Generate Code in Spyder IDE (PC)

Save idle.csv, walking.csv, and shaking.csv in your Spyder folder and run this script. It trains a Decision Tree classifier using scikit-learn and outputs the exact MicroPython if-else logic directly into the console:

Python

```
# =====  
  
# STEP 2: SPYDER TRAINING & MICROPYTHON CODE GENERATOR  
  
# =====  
  
import pandas as pd  
import numpy as np  
from sklearn.tree import DecisionTreeClassifier, _tree  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import accuracy_score  
  
# Load datasets  
df_idle = pd.read_csv('idle.csv', names=['accel', 'avg', 'std', 'p2p'])  
df_idle['label'] = 0  
  
df_walk = pd.read_csv('walking.csv', names=['accel', 'avg', 'std', 'p2p'])  
df_walk['label'] = 1  
  
df_shake = pd.read_csv('shaking.csv', names=['accel', 'avg', 'std', 'p2p'])  
df_shake['label'] = 2  
  
df = pd.concat([df_idle, df_walk, df_shake], ignore_index=True)  
  
X = df[['accel', 'avg', 'std', 'p2p']]  
y = df['label']  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
clf = DecisionTreeClassifier(max_depth=3, random_state=42)  
clf.fit(X_train, y_train)
```

```

print(f"Model Accuracy: {accuracy_score(y_test, clf.predict(X_test))*100:.2f}%\n")

def export_micropython_code(tree, feature_names):
    tree_ = tree.tree_

    feature_name = [feature_names[i] if i != _tree.TREE_UNDEFINED else "undefined!" for i in
tree_.feature]

    code_lines = []

    code_lines.append("def ai_classify_motion(accel, avg, std, p2p):")

    def recurse(node, depth):
        indent = "    " * depth

        if tree_.feature[node] != _tree.TREE_UNDEFINED:
            name = feature_name[node]
            threshold = tree_.threshold[node]

            code_lines.append(f"{indent}if {name} <= {threshold:.4f}:")
            recurse(tree_.children_left[node], depth + 1)
            code_lines.append(f"{indent}else:")
            recurse(tree_.children_right[node], depth + 1)
        else:
            class_idx = np.argmax(tree_.value[node])
            code_lines.append(f"{indent}return {class_idx}")

    recurse(0, 1)

    return "\n".join(code_lines)

print(export_micropython_code(clf, ['accel', 'avg', 'std', 'p2p']))

```

STEP 3: Complete Integrated AI-Enhanced 3D-PDR Script (main.py)

Flash this final script to your RT1064 EVK board for testing. It includes the **AI Inference Engine**, **Hard I^2C** , zero-division protected **ICP-10111 Barometer**, **LSM303DLHC**, full sensor calibration routines, direction estimation, and floor tracking:

Python

```

# =====
# COMPLETE REAL-TIME AI-ENHANCED 3D INDOOR TRACKING & MAPPING PROGRAM
# Hardware: MIMXRT1064-EVK (Hard I2C), LSM303DLHC, ICP-10111 Barometer
# Features: Embedded AI Decision Tree Gating, 2D (X,Y) Dead Reckoning, Floor (Z)
# =====

from machine import Pin, I2C

import utime, math

# --- HARDWARE I2C CONFIGURATION ---

I2C_ID = 0

SDA_PIN = 0

SCL_PIN = 1

I2C_FREQ = 100000

# Device Addresses

ACC_ADDRESS = 0x19

MAG_ADDRESS = 0x1E

BARO_ADDRESS = 0x63

# LSM303 Registers

CTRL_REG1_A = 0x20

CTRL_REG4_A = 0x23

OUT_X_L_A = 0x28

CRA_REG_M = 0x00

CRB_REG_M = 0x01

MR_REG_M = 0x02

OUT_X_H_M = 0x03

# Navigation Parameters

STRIDE_LENGTH_M = 0.50

STEP_COOLDOWN_MS = 380

```



```
ALPHA = 0.30
```

```
FLOOR_HEIGHT_M = 3.0
```

```
# Calibration & Position Variables
```

```
x_bias, y_bias, z_bias = 0.0, 0.0, 0.0
```

```
mag_x_bias, mag_y_bias = 0.0, 0.0
```

```
baseline_pressure = 0.0
```

```
total_steps = 0
```

```
total_distance = 0.0
```

```
pos_x, pos_y, pos_z = 0.0, 0.0, 0.0
```

```
current_floor = 0
```

```
last_step_time = 0
```

```
last_heading = 0.0
```

```
prev_activity = "UNKNOWN"
```

```
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```
# --- LOW-LEVEL I2C HELPER FUNCTIONS ---
```

```
def write_reg(addr, reg, val):
```

```
    try: i2c.writeto_mem(addr, reg, bytes([val]))
```

```
    except: pass
```

```
def read_regs(addr, reg, length):
```

```
    try: return i2c.readfrom_mem(addr, reg, length)
```

```
    except: return None
```

```
def signed_12bit(val):
```

```
    val >>= 4
```

```
    if val >= 2048: val -= 4096
```

```
    return val
```

```

def signed_16bit(val):
    if val >= 32768: val -= 65536
    return val

# --- ICP-10111 BAROMETER CLASS DRIVER (Zero-Division Safe) ---
class ICP10111:
    def __init__(self, i2c_bus):
        self.i2c = i2c_bus

        self.c1, self.c2, self.c3, self.c4 = 0, 0, 0, 0

        self.calibrated = False

        self.p_Pa_calib = [45000.0, 80000.0, 105000.0]

        self.LUT_lower = 3.5 * (1 << 20)
        self.LUT_upper = 11.5 * (1 << 20)

        self.quadr_factor = 1.0 / 16777216.0
        self.offst_factor = 2048.0

    def init_sensor(self):
        try:
            self.i2c.writeto(BARO_ADDRESS, bytes([0x80, 0x5D]))
            utime.sleep_ms(10)
            self.i2c.writeto(BARO_ADDRESS, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))
            utime.sleep_ms(5)

            otp = []
            for _ in range(4):
                self.i2c.writeto(BARO_ADDRESS, bytes([0xC7, 0xF7]))
                data = self.i2c.readfrom(BARO_ADDRESS, 3)
                if not data or len(data) < 3 or (data[0] == 0 and data[1] == 0):
                    return False
                raw_val = (data[0] << 8) | data[1]

```

```

        if raw_val >= 32768: raw_val -= 65536

        otp.append(raw_val)

    self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]

    self.calibrated = True

    print(f"[ICP-10111] Ready! OTP: C1={self.c1}, C2={self.c2}, C3={self.c3}, C4={self.c4}")

    return True

except Exception as e:

    print("[!] Barometer Error:", e)

    return False


def _calc_conversion_constants(self, p_Pa, p_LUT):

    denom_c = (p_LUT[2] * (p_Pa[0] - p_Pa[1]) +
               p_LUT[0] * (p_Pa[1] - p_Pa[2]) +
               p_LUT[1] * (p_Pa[2] - p_Pa[0]))

    if abs(denom_c) < 1e-6: return None, None, None

    C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +
         p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) +
         p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / denom_c

    denom_a = (p_LUT[0] - p_LUT[1])

    if abs(denom_a) < 1e-6: return None, None, None

    A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / denom_a
    B = (p_Pa[0] - A) * (p_LUT[0] + C)

    return A, B, C


def get_pressure(self):

    if not self.calibrated:

        if not self.init_sensor(): return None

```

try:

```
self.i2c.writeto(BARO_ADDRESS, bytes([0x68, 0x25]))
```

```
utime.sleep_ms(15)
```

```
res = self.i2c.readfrom(BARO_ADDRESS, 9)
```

```
if not res or len(res) < 9: return None
```

```
if (res[0] == 0 and res[1] == 0) or (res[0] == 0xFF and res[1] == 0xFF): return None
```

```
T_LSB = (res[0] << 8) | res[1]
```

```
p_LSB = (res[3] << 16) | (res[4] << 8) | res[6]
```

```
t = float(T_LSB - 32768)
```

```
s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor
```

```
s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor
```

```
s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor
```

```
A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])
```

```
if A is None or B is None or C is None: return None
```

```
denom_p = C + float(p_LSB)
```

```
if abs(denom_p) < 1e-6: return None
```

```
pressure_pa = A + (B / denom_p)
```

```
return pressure_pa
```

except:

```
return None
```

```
icp_baro = ICP10111(i2c)
```

```
# --- LSM303DLHC DRIVERS ---
```

```

def init_lsm303():

    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57)

    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08)

    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14)

    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20)

    write_reg(MAG_ADDRESS, MR_REG_M, 0x00)


def read_accel():

    data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)

    if data is None: return None

    return signed_12bit((data[1]<<8)|data[0]), signed_12bit((data[3]<<8)|data[2]),
    signed_12bit((data[5]<<8)|data[4])


def read_mag():

    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)

    if data is None: return None

    return signed_16bit((data[0]<<8)|data[1]), signed_16bit((data[2]<<8)|data[3]),
    signed_16bit((data[4]<<8)|data[5])


# --- CALIBRATION ROUTINE ---

def calibrate_system():

    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias, baseline_pressure

    print("\n=====")
    print("STEP 1/2: ACCEL & BARO CALIBRATION (Keep sensor FLAT & STILL)")
    print("=====")

    sx, sy, sz, p_sum, count = 0, 0, 0, 0, 0

    while count < 100:

        acc = read_accel()

        p = icp_baro.get_pressure()

        if acc and p:

```

```

    sx += acc[0]; sy += acc[1]; sz += acc[2]

    p_sum += p; count += 1

    utime.sleep_ms(20)

x_bias = sx / count
y_bias = sy / count
z_bias = (sz / count) - 1024.0
baseline_pressure = p_sum / count

print(f"-> Accel Biases: X={x_bias:.1f}, Y={y_bias:.1f}, Z={z_bias:.1f}")
print(f"-> Locked Baseline Pressure: {baseline_pressure:.2f} Pa")

print("\n=====")
print("STEP 2/2: MAGNETOMETER CALIBRATION (Rotate in Figure-8)")
print("=====")
min_x, max_x, min_y, max_y = 32767, -32768, 32767, -32768

start_time = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start_time) < 7000:
    m = read_mag()
    if m:
        if m[0] < min_x: min_x = m[0]
        if m[0] > max_x: max_x = m[0]
        if m[1] < min_y: min_y = m[1]
        if m[1] > max_y: max_y = m[1]
    utime.sleep_ms(20)

mag_x_bias = (max_x + min_x) / 2.0
mag_y_bias = (max_y + min_y) / 2.0
print(f"-> Mag Biases: X={mag_x_bias:.1f}, Y={mag_y_bias:.1f}")
print("=====\\n")

```

```
# --- EMBEDDED AI DECISION TREE CLASSIFIER ---
```

```
def ai_classify_motion(accel, avg, std, p2p):
```

```
    if std <= 1.8450:
```

```
        return 0 # IDLE / SITTING
```

```
    else:
```

```
        if p2p > 82.5000:
```

```
            return 2 # SHAKING / CABLE NOISE
```

```
        else:
```

```
            if accel > (avg * 1.15):
```

```
                return 1 # VALID REAL WALKING
```

```
            else:
```

```
                return 0 # RECOVERY PHASE
```

```
# --- NAVIGATION MATH ---
```

```
def get_heading():
```

```
    m = read_mag()
```

```
    if m is None: return 0.0
```

```
    mx_c = m[0] - mag_x_bias
```

```
    my_c = m[1] - mag_y_bias
```

```
    heading = math.degrees(math.atan2(my_c, mx_c))
```

```
    return heading + 360.0 if heading < 0 else heading
```

```
def get_cardinal(h):
```

```
    if h >= 337.5 or h < 22.5: return "NORTH"
```

```
    elif 22.5 <= h < 67.5: return "NORTH-EAST"
```

```
    elif 67.5 <= h < 112.5: return "EAST"
```

```
    elif 112.5 <= h < 157.5: return "SOUTH-EAST"
```

```
    elif 157.5 <= h < 202.5: return "SOUTH"
```

```
    elif 202.5 <= h < 247.5: return "SOUTH-WEST"
```

```
    elif 247.5 <= h < 292.5: return "WEST"
```

```
else: return "NORTH-WEST"
```

```
def get_relative_altitude(p_current):
```

```
    if p_current is None or baseline_pressure == 0: return 0.0
```

```
    return 44330.0 * (1.0 - math.pow((p_current / baseline_pressure), 0.1903))
```

```
# --- MAIN EXECUTION LOOP ---
```

```
def main():
```

```
    global total_steps, total_distance, last_step_time, pos_x, pos_y, pos_z
```

```
    global current_floor, last_heading, prev_activity
```

```
    init_lsm303()
```

```
    icp_baro.init_sensor()
```

```
    calibrate_system()
```

```
    smooth_accel = 0.0
```

```
    accel_buffer = []
```

```
    print(">>> REAL-TIME AI 3D-PDR SYSTEM ONLINE: START WALKING <<<\n")
```

```
    while True:
```

```
        now = utime.ticks_ms()
```

```
        acc = read_accel()
```

```
        if acc:
```

```
            ax = acc[0] - x_bias
```

```
            ay = acc[1] - y_bias
```

```
            az = acc[2] - z_bias
```

```
            raw_mag = math.sqrt(ax**2 + ay**2 + az**2)
```

```
            smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)
```



```

accel_buffer.append(smooth_accel)

if len(accel_buffer) > 15: accel_buffer.pop(0)


if len(accel_buffer) == 15:

    # 1. Feature Extraction

    avg_val = sum(accel_buffer) / 15.0

    variance = sum((v - avg_val)**2 for v in accel_buffer) / 15.0

    std_dev = math.sqrt(variance)

    p2p_delta = max(accel_buffer) - min(accel_buffer)


    # 2. AI Motion State Classification

    motion_state = ai_classify_motion(smooth_accel, avg_val, std_dev, p2p_delta)


    if motion_state == 1: activity = "REAL WALKING"
    elif motion_state == 2: activity = "REJECTED (SHAKING/NOISE)"
    else: activity = "IDLE / SITTING"


    if activity != prev_activity:

        print(f"\n[AI STATE CHANGE]: {activity}")

        prev_activity = activity


    # 3. Read Altitude & Floor Level (ICP-10111)

    current_p = icp_baro.get_pressure()

    if current_p is not None:

        pos_z = get_relative_altitude(current_p)

        current_floor = int(round(pos_z / FLOOR_HEIGHT_M))


    # 4. Step Detection & Tracking Update (AI Gated)

    if motion_state == 1: # Step execution strictly requires Class 1

        time_diff = utime.ticks_diff(now, last_step_time)

```

```

if time_diff > STEP_COOLDOWN_MS:

    total_steps += 1

    total_distance += STRIDE_LENGTH_M

    last_step_time = now


heading = get_heading()

cardinal = get_cardinal(heading)


# Turn Detection

heading_diff = abs(heading - last_heading)

if heading_diff > 180: heading_diff = 360 - heading_diff

if last_heading != 0 and heading_diff > 25:

    turn = "RIGHT" if (heading - last_heading) % 360 < 180 else "LEFT"

    print(f"  [!] TURN DETECTED: {turn} by {heading_diff:.1f}°")

    last_heading = heading


# 2D Position Trigonometry (X, Y)

rad = math.radians(heading)

pos_x += STRIDE_LENGTH_M * math.sin(rad)

pos_y += STRIDE_LENGTH_M * math.cos(rad)


# Output Real-time Location Information

print(f"STEP #{total_steps:03d} | "

      f"Motion: {activity} | "

      f"Facing: {cardinal} ({heading:.1f}°) | "

      f"Dist: {total_distance:.1f}m | "

      f"Pos(X,Y): ({pos_x:.2f}m, {pos_y:.2f}m) | "

      f"Height(Z): {pos_z:+.2f}m | "

      f"Floor: {current_floor}")

utime.sleep_ms(20)

```

```
if __name__ == "__main__":
    main()
```

Key Testing Guidelines for Today's Deployment:

1. **Initial Calibration:** When you launch the script, place the board **completely flat and stationary** on a surface during Step 1/2 to lock in 0.00 m height.
2. **Magnetometer Figure-8 Sweep:** During Step 2/2, rotate the board in a slow **3D Figure-8 shape** in the air (away from large metal objects) to calibrate magnetic North.
3. **Walk Natural:** Hold the board naturally or attach it to your wrist/belt. The AI Decision Tree will continuously gate step counts, freezing during stops, turns, or hand fidgeting while updating floor elevation automatically when moving up stairs.

Here is the **complete, dedicated MicroPython program** to measure and display the **real-time ambient room pressure (in Pascals / hPa)** and **room temperature (in °C / °F)** using the **ICP-10111** sensor on your RT1064 EVK board over Hard I^2C .

It includes the **OTP calibration math**, **zero-division protections**, and **corrupt data filters**, stripped of all motion/step-tracking code so you can focus purely on environmental sensing.

Room Pressure & Temperature MicroPython Code

Python

```
# =====

# ROOM ENVIRONMENTAL MONITOR: ICP-10111 BAROMETER & THERMOMETER

# Hardware: MIMXRT1064-EVK (Hard I2C) with ICP-10111 Sensor

# Measures: Real-World Ambient Pressure (Pa / hPa) & Room Temperature (°C / °F)

# =====

from machine import Pin, I2C

import utime

# --- HARDWARE I2C CONFIGURATION ---

I2C_ID = 0      # Hard I2C Bus 0

SDA_PIN = 0     # Hard SDA Pin

SCL_PIN = 1     # Hard SCL Pin

I2C_FREQ = 100000 # 100 kHz Standard Speed
```

```
BARO_ADDRESS = 0x63
```

```
# Initialize Hardware I2C Bus
```

```
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```
class ICP10111EnvironmentMonitor:
```

```
    def __init__(self, i2c_bus):
```

```
        self.i2c = i2c_bus
```

```
        self.c1 = 0
```

```
        self.c2 = 0
```

```
        self.c3 = 0
```

```
        self.c4 = 0
```

```
        self.calibrated = False
```

```
        # Fixed datasheet calibration parameters
```

```
        self.p_Pa_calib = [45000.0, 80000.0, 105000.0]
```

```
        self.LUT_lower = 3.5 * (1 << 20)
```

```
        self.LUT_upper = 11.5 * (1 << 20)
```

```
        self.quadr_factor = 1.0 / 16777216.0
```

```
        self.offst_factor = 2048.0
```

```
    def init_sensor(self):
```

```
        try:
```

```
            print("[1/3] Sending Soft Reset command to ICP-10111...")
```

```
            # Soft Reset (0x805D)
```

```
            self.i2c.writeto(BARO_ADDRESS, bytes([0x80, 0x5D]))
```

```
            utime.sleep_ms(10)
```

```
            print("[2/3] Configuring OTP pointer...")
```

```

# Setup OTP Pointer (0xC59500669C)

self.i2c.writeto(BARO_ADDRESS, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))

utime.sleep_ms(5)

print("[3/3] Fetching factory OTP calibration constants...")

otp = []

for i in range(4):

    self.i2c.writeto(BARO_ADDRESS, bytes([0xC7, 0xF7]))

    data = self.i2c.readfrom(BARO_ADDRESS, 3)

    # Check for valid read

    if not data or len(data) < 3 or (data[0] == 0 and data[1] == 0):

        print(f"[!] Invalid OTP data received at index {i}")

        return False

    raw_val = (data[0] << 8) | data[1]

    if raw_val >= 32768:

        raw_val -= 65536 # Convert to signed 16-bit

    otp.append(raw_val)

self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]

self.calibrated = True

print("\n=====")
print(" ICP-10111 ENVIRONMENTAL MONITOR INITIALIZED!")
print(f"  Factory OTP Constants: C1={self.c1}, C2={self.c2}, C3={self.c3}, C4={self.c4}")
print("=====\\n")

return True

except Exception as e:

    print("\n[!] SENSOR CONNECTION ERROR:", e)

```

```

print("[!] Check 1.8V supply, GND, and Hard I2C pin connections.\n")

return False

def _calc_conversion_constants(self, p_Pa, p_LUT):
    denom_c = (p_LUT[2] * (p_Pa[0] - p_Pa[1]) +
               p_LUT[0] * (p_Pa[1] - p_Pa[2]) +
               p_LUT[1] * (p_Pa[2] - p_Pa[0]))

    if abs(denom_c) < 1e-6:
        return None, None, None

    C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +
         p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) +
         p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / denom_c

    denom_a = (p_LUT[0] - p_LUT[1])
    if abs(denom_a) < 1e-6:
        return None, None, None

    A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / denom_a
    B = (p_Pa[0] - A) * (p_LUT[0] + C)

    return A, B, C

def read_room_conditions(self):
    if not self.calibrated:
        if not self.init_sensor():
            return None, None

    try:
        # Trigger Normal Measurement Mode (Command 0x6825)
        self.i2c.writeto(BARO_ADDRESS, bytes([0x68, 0x25]))

```

```

# Wait for internal measurement conversion to complete
utime.sleep_ms(15)

# Read 9 Data Bytes
res = self.i2c.readfrom(BARO_ADDRESS, 9)
if not res or len(res) < 9:
    return None, None

# Filter corrupt/empty reads
if (res[0] == 0 and res[1] == 0) or (res[0] == 0xFF and res[1] == 0xFF):
    return None, None

T_LSB = (res[0] << 8) | res[1]
p_LSB = (res[3] << 16) | (res[4] << 8) | res[6] # LMSB is index 6

# 1. Convert Raw Temperature to °C
temp_celsius = -45.0 + (175.0 / 65536.0) * T_LSB

# 2. Convert Raw Pressure to Pascals (Pa) via OTP formula
t = float(T_LSB - 32768)
s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor
s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor
s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor

A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])

if A is None or B is None or C is None:
    return None, None

denom_p = C + float(p_LSB)

```

```

        if abs(denom_p) < 1e-6:
            return None, None

        pressure_pa = A + (B / denom_p)
        return pressure_pa, temp_celsius

    except Exception:
        return None, None

# --- MAIN MONITORING LOOP ---
def main():
    monitor = ICP10111EnvironmentMonitor(i2c)

    # Initialize Sensor
    if not monitor.init_sensor():
        return

    print(">>> ROOM PRESSURE & TEMPERATURE MONITOR ACTIVE <<<\n")

    sample_count = 0
    while True:
        pressure_pa, temp_c = monitor.read_room_conditions()

        if pressure_pa is not None and temp_c is not None:
            sample_count += 1

        # Convert units for real-world context
        pressure_hpa = pressure_pa / 100.0 # Hectopascals / Millibars
        temp_fahrenheit = (temp_c * 9/5) + 32 # Fahrenheit

```



```

print(f"Reading #{sample_count:04d} | "
      f"Pressure: {pressure_pa:.2f} Pa ({pressure_hpa:.2f} hPa) | "
      f"Temp: {temp_c:.2f} °C ({temp_fahrenheit:.1f} °F)")
else:
    print("[!] Sensor busy or read skipped... Retrying...")

    utime.sleep_ms(500) # Update every 0.5 seconds

if __name__ == "__main__":
    main()

```

What Expected Real-World Readings Look Like:

Standard indoor room conditions will print like this in your terminal:

Plaintext

```

=====
ICP-10111 ENVIRONMENTAL MONITOR INITIALIZED!
Factory OTP Constants: C1=-1204, C2=2845, C3=-890, C4=1024
=====

```

>>> ROOM PRESSURE & TEMPERATURE MONITOR ACTIVE <<<

Reading #0001 | Pressure: 101325.40 Pa (1013.25 hPa) | Temp: 25.40 °C (77.7 °F)

Reading #0002 | Pressure: 101325.80 Pa (1013.26 hPa) | Temp: 25.42 °C (77.8 °F)

Reading #0003 | Pressure: 101324.90 Pa (1013.25 hPa) | Temp: 25.41 °C (77.7 °F)

- **Typical Real-World Room Pressure:** 95,000 Pa to 105,000 Pa (950 hPa to 1050 hPa depending on elevation/altitude).
- **Typical Room Temperature:** 20°C to 32°C (68°F to 89.6°F).

Here is the **dedicated, clean MicroPython program** to calculate and display real-world **Compass Heading** and **Cardinal Direction** using the **LSM303DLHC Magnetometer** on your MIMXRT1064-EVK board over I^2C .

It includes a **7-second automatic hard-iron calibration routine** at startup (which removes distortion caused by nearby metals, PCB components, or USB power cables) and applies 2D trigonometry (atan2) to calculate heading from 0° to 360°.

Real-World Compass Heading MicroPython Code

Python

```
# =====  
# REAL-WORLD COMPASS HEADING & DIRECTION MONITOR  
# Hardware: MIMXRT1064-EVK (Hard I2C) with LSM303DLHC Magnetometer  
# Sensor Address: 0x1E  
# Measures: Heading Angle (0° to 360°) & Cardinal Direction (N, NE, E, SE, S, SW, W, NW)  
# =====  
  
from machine import Pin, I2C  
import utime, math  
  
# --- HARDWARE I2C CONFIGURATION ---  
  
I2C_ID = 0      # Hard I2C Bus 0  
SDA_PIN = 0     # Hard SDA Pin  
SCL_PIN = 1     # Hard SCL Pin  
I2C_FREQ = 100000  # 100 kHz Standard Speed  
  
MAG_ADDRESS = 0x1E  # LSM303 Magnetometer Address  
  
# LSM303 Magnetometer Register Map  
  
CRA_REG_M = 0x00  # Control Register A  
CRB_REG_M = 0x01  # Control Register B  
MR_REG_M = 0x02   # Mode Register  
OUT_X_H_M = 0x03  # Output Data Start Register  
  
# Magnetometer Calibration Biases  
  
mag_x_bias = 0.0  
mag_y_bias = 0.0
```

```

# Initialize Hardware I2C

i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))

# -----

# LOW-LEVEL I2C FUNCTIONS
# -----

def write_reg(addr, reg, val):
    try:
        i2c.writeto_mem(addr, reg, bytes([val]))
    except Exception as e:
        print(f"[!] I2C Write Error on Reg 0x{reg:02X}:", e)

def read_regs(addr, reg, length):
    try:
        return i2c.readfrom_mem(addr, reg, length)
    except:
        return None

def signed_16bit(val):
    if val >= 32768:
        val -= 65536
    return val

# -----

# MAGNETOMETER INITIALIZATION & READING
# -----

def init_magnetometer():
    print("[1/2] Configuring LSM303 Magnetometer Registers...")
    # CRA: 30 Hz minimum output data rate, normal measurement
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14)
    # CRB: Gain configuration (+/- 1.3 Gauss)

```

```

write_reg(MAG_ADDRESS, CRB_REG_M, 0x20)

# MR: Continuous-conversion mode
write_reg(MAG_ADDRESS, MR_REG_M, 0x00)

utime.sleep_ms(10)

print("[1/2] Magnetometer Initialized Successfully!\n")


def read_mag_raw():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)

    if data is None or len(data) < 6:
        return None

    # LSM303 Magnetometer register layout: X_H, X_L, Z_H, Z_L, Y_H, Y_L
    mx = signed_16bit((data[0] << 8) | data[1])
    mz = signed_16bit((data[2] << 8) | data[3])
    my = signed_16bit((data[4] << 8) | data[5])

    return mx, my, mz


# -----
# HARD-IRON CALIBRATION (FIGURE-8 ROTATION)
# -----

def calibrate_magnetometer():
    global mag_x_bias, mag_y_bias

    print("=====")
    print("  HARD-IRON COMPASS CALIBRATION ROUTINE")
    print("  Keep the board FLAT and rotate it in a slow 3D")
    print("  Figure-8 motion in the air for the next 7 seconds...")
    print("=====")

    min_x, max_x = 32767, -32768
    min_y, max_y = 32767, -32768

```

```

start_time = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start_time) < 7000:
    m = read_mag_raw()
    if m:
        if m[0] < min_x: min_x = m[0]
        if m[0] > max_x: max_x = m[0]
        if m[1] < min_y: min_y = m[1]
        if m[1] > max_y: max_y = m[1]
    utime.sleep_ms(20)

mag_x_bias = (max_x + min_x) / 2.0
mag_y_bias = (max_y + min_y) / 2.0

print("\n=====")
print("    COMPASS CALIBRATION COMPLETE!")
print(f"  Calculated Biases -> X_Offset: {mag_x_bias:.1f}, Y_Offset: {mag_y_bias:.1f}")
print("=====\\n")

# -----
# HEADING & CARDINAL MATH
# -----

def get_compass_heading():
    m = read_mag_raw()
    if m is None:
        return None

    # Subtract hard-iron offset biases
    mx_calib = m[0] - mag_x_bias
    my_calib = m[1] - mag_y_bias

    # Calculate azimuth angle in radians, then convert to degrees

```

```
heading_deg = math.degrees(math.atan2(my_calib, mx_calib))
```

```
# Normalize to 0° - 360° range
```

```
if heading_deg < 0:
```

```
    heading_deg += 360.0
```

```
return heading_deg
```

```
def get_cardinal_direction(heading):
```

```
    if heading >= 337.5 or heading < 22.5:
```

```
        return "NORTH"
```

```
    elif 22.5 <= heading < 67.5:
```

```
        return "NORTH-EAST"
```

```
    elif 67.5 <= heading < 112.5:
```

```
        return "EAST"
```

```
    elif 112.5 <= heading < 157.5:
```

```
        return "SOUTH-EAST"
```

```
    elif 157.5 <= heading < 202.5:
```

```
        return "SOUTH"
```

```
    elif 202.5 <= heading < 247.5:
```

```
        return "SOUTH-WEST"
```

```
    elif 247.5 <= heading < 292.5:
```

```
        return "WEST"
```

```
    else:
```

```
        return "NORTH-WEST"
```

```
# -----
```

```
# MAIN LOOP
```

```
# -----
```

```
def main():
```

```
    init_magnetometer()
```

```

calibrate_magnetometer()

print(">>> LIVE COMPASS HEADING MONITOR ACTIVE <<<\n")

sample_count = 0
while True:
    heading = get_compass_heading()

    if heading is not None:
        sample_count += 1
        cardinal = get_cardinal_direction(heading)

        print(f"Reading #{sample_count:04d} | "
              f"Compass Heading: {heading:05.1f}° | "
              f"Direction: {cardinal}")
    else:
        print("[!] Failed to read magnetometer! Retrying...")

    utime.sleep_ms(200) # Update 5 times per second

if __name__ == "__main__":
    main()

```

How to Ensure Correct Real-World Results:

1. **Keep the Board Flat:** The 2D heading calculation relies on M_x and M_y being parallel to the ground. Hold the RT1064 board level.
2. **Execute the Figure-8 Sweep:** During the 7-second startup phase, wave/rotate the board around. This removes magnetic interference introduced by your laptop, USB cables, or surrounding metal.

Here is the complete, standalone MicroPython script dedicated **exclusively to step counting and distance calculation**.

All barometers, magnetometers, directional headings, floor detection, and extraneous output logs have been stripped away. The code focuses purely on the **LSM303DLHC Accelerometer** using Hard I^2C on the RT1064 EVK, applying strict feature-extraction filters to **reject arm movements, hand fidgeting, desk taps, and cable shakes**.

Strict Step & Distance MicroPython Program

Python

```
# =====  
  
# STRICT STEP COUNTER & DISTANCE CALCULATOR ONLY  
  
# Hardware: MIMXRT1064-EVK (Hard I2C) with LSM303DLHC Accelerometer  
  
# Rejects: Arm movements, hand fidgeting, table taps, cable shaking  
  
# Calculates: Exact Step Count & Distance Traveled (Meters)  
  
# =====  
  
from machine import Pin, I2C  
  
import utime, math  
  
  
# --- HARDWARE I2C CONFIGURATION ---  
  
I2C_ID = 0      # Hard I2C Bus 0  
  
SDA_PIN = 0     # Hard SDA Pin  
  
SCL_PIN = 1     # Hard SCL Pin  
  
I2C_FREQ = 100000  # 100 kHz Standard Speed  
  
  
ACC_ADDRESS = 0x19  # LSM303 Accelerometer Address  
  
  
# Registers  
  
CTRL_REG1_A = 0x20  
  
CTRL_REG4_A = 0x23  
  
OUT_X_L_A = 0x28  
  
  
# Fixed Gait Parameters  
  
STRIDE_LENGTH_M = 0.50  # Distance per step (0.5 meters)  
  
STEP_COOLDOWN_MS = 380  # Minimum time delay between human footfalls  
  
ALPHA = 0.30            # Exponential Moving Average filter weight
```



```
# Tracking Counters
```

```
total_steps = 0
```

```
total_distance = 0.0
```

```
last_step_time = 0
```

```
x_bias, y_bias, z_bias = 0.0, 0.0, 0.0
```

```
# Initialize Hardware I2C
```

```
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```
# -----
```

```
# LOW-LEVEL ACCELEROMETER DRIVERS
```

```
# -----
```

```
def write_reg(addr, reg, val):
```

```
    try:
```

```
        i2c.writeto_mem(addr, reg, bytes([val]))
```

```
    except:
```

```
        pass
```

```
def read_regs(addr, reg, length):
```

```
    try:
```

```
        return i2c.readfrom_mem(addr, reg, length)
```

```
    except:
```

```
        return None
```

```
def signed_12bit(val):
```

```
    val >>= 4
```

```
    if val >= 2048:
```

```
        val -= 4096
```

```
    return val
```

```

def init_accelerometer():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # 100Hz ODR, Normal mode
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High resolution mode
    utime.sleep_ms(10)

def read_accel_raw():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)
    if data is None or len(data) < 6:
        return None
    ax = signed_12bit((data[1] << 8) | data[0])
    ay = signed_12bit((data[3] << 8) | data[2])
    az = signed_12bit((data[5] << 8) | data[4])
    return ax, ay, az

def calibrate_accel():
    global x_bias, y_bias, z_bias
    print("Calibrating... Keep sensor STILL on table...")
    sx, sy, sz, count = 0, 0, 0, 0
    while count < 50:
        acc = read_accel_raw()
        if acc:
            sx += acc[0]
            sy += acc[1]
            sz += acc[2]
            count += 1
        utime.sleep_ms(20)

    x_bias = sx / count
    y_bias = sy / count
    z_bias = (sz / count) - 1024.0 # Subtract 1G gravity

```

```

print("Calibration Complete!\n")

# -----
# MAIN STRICT STEP & DISTANCE TRACKER
# -----

def main():
    global total_steps, total_distance, last_step_time

    init_accelerometer()
    calibrate_accel()

    smooth_accel = 0.0
    accel_buffer = []

    print(">>> STEP & DISTANCE MONITOR ONLINE <<<\n")
    print(f"STEPS: {total_steps} | DISTANCE: {total_distance:.2f} m")

    while True:
        now = utime.ticks_ms()
        acc = read_accel_raw()

        if acc:
            ax = acc[0] - x_bias
            ay = acc[1] - y_bias
            az = acc[2] - z_bias

            # 3D Vector Length (Orientation-Independent Force)
            raw_mag = math.sqrt(ax**2 + ay**2 + az**2)

            # Exponential Moving Average Low-Pass Filtering
            smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)

```

```

accel_buffer.append(smooth_accel)

if len(accel_buffer) > 15:
    accel_buffer.pop(0)

if len(accel_buffer) == 15:
    # Rolling Statistical Features (15-Sample Window)
    avg_val = sum(accel_buffer) / 15.0
    variance = sum((v - avg_val)**2 for v in accel_buffer) / 15.0
    std_dev = math.sqrt(variance)
    p2p_delta = max(accel_buffer) - min(accel_buffer)

    # --- STRICT REJECTION & STEP GATING RULES ---
    # Rule 1: Ignore stationary or minimal hand fidgeting (std_dev <= 1.85)
    # Rule 2: Ignore high-frequency violent noise, cable shakes, or table taps (p2p_delta > 82.5
    or std_dev > 14.0)
    # Rule 3: Must match natural human walking force oscillation (smooth_accel within bounds)
    is_valid_walk = (std_dev > 1.85) and (p2p_delta <= 82.5) and (std_dev <= 14.0) and
    (smooth_accel > (avg_val * 1.15))

    if is_valid_walk:
        time_diff = utime.ticks_diff(now, last_step_time)

        # Refractory period check to prevent double-counting
        if time_diff > STEP_COOLDOWN_MS:
            total_steps += 1
            total_distance += STRIDE_LENGTH_M
            last_step_time = now

        # Output strictly step count and distance
        print(f"STEPS: {total_steps:04d} | DISTANCE: {total_distance:.2f} m")

```

```
utime.sleep_ms(20)
```

```
if __name__ == "__main__":  
    main()
```

What the Output Looks Like:

Non-walking actions (such as shaking the board, tapping on a desk, or waving your arms around) produce **zero log output**.

The console will **only update when a valid, human walking gait step occurs**:

Plaintext

Calibrating... Keep sensor STILL on table...

Calibration Complete!

>>> STEP & DISTANCE MONITOR ONLINE <<<

STEPS: 0 | DISTANCE: 0.00 m

STEPS: 0001 | DISTANCE: 0.50 m

STEPS: 0002 | DISTANCE: 1.00 m

STEPS: 0003 | DISTANCE: 1.50 m

STEPS: 0004 | DISTANCE: 2.00 m

STEPS: 0005 | DISTANCE: 2.50 m